NumPy & Pandas for Data Analytics

Assignment Report

Submitted by: Deepanshu

Tools: Python 3, NumPy, Pandas

Task	Topic	Marks
Task 1	NumPy Introduction & Arrays	10
Task 2	NumPy Indexing, Slicing & Reshaping	10
Task 3	NumPy Mathematical & Statistical Operations	10
Task 4	Pandas Series & DataFrame	10
Task 5	Reading & Inspecting Data	10
Task 6	Selecting, Filtering & Sorting Data	10
Task 7	Handling Missing Values	10
Task 8	Merge, Concatenate, GroupBy & Pivot Table	15
Task 9	Exporting Data	5
Task 10	Mini Data Analysis Project: Sales Performance Analysis	10
	Total	100

Datasets Used

Tasks 5 to 10 use a Sales dataset made up of three related CSV files, generated with **generate_dataset.py** (fixed random seed, so results are reproducible). The data is synthetic and for practice only. **sales_data.csv** (600 orders) deliberately contains missing values in Region, Quantity, Discount, Payment_Mode and Rating so that Task 7 has real gaps to handle.

sales_data.csv shape = (600, 10)

Orders: Order_ID, Order_Date, Customer_ID, Product_ID, Region, Quantity, Unit_Price, Discount, Payment_Mode, Rating

	Order_ID	Order_Date	Customer_ID	Product_ID	Region	Quantity	Unit_Price	Discount	Payment_Mode	Rating
0	ORD0001	2025-01-02	C057	P12	West	2.0	370.0	NaN	Credit Card	3.0
1	ORD0002	2025-01-02	C076	P03	West	6.0	2186.0	0.20	UPI	3.0
2	ORD0003	2025-01-02	C012	P08	South	4.0	4595.0	0.00	Credit Card	NaN
3	ORD0004	2025-01-02	C043	P05	Central	7.0	5159.0	0.15	Credit Card	4.0
4	ORD0005	2025-01-02	C003	P02	Central	1.0	19975.0	0.00	Credit Card	1.0

products.csv shape = (12, 4)

Product master: Product_ID, Product_Name, Category, Cost_Price

	Product_ID	Product_Name	Category	Cost_Price
0	P01	Laptop	Electronics	42000
1	P02	Smartphone	Electronics	15000
2	P03	Headphones	Electronics	1500
3	P04	Smartwatch	Electronics	4500
4	P05	Office Chair	Furniture	4200

customers.csv shape = (80, 2)

Customer master: Customer_ID, Segment

	Customer_ID	Segment
0	C001	Corporate
1	C002	Home Office
2	C003	Consumer
3	C004	Consumer
4	C005	Corporate

Missing values in sales_data.csv

Order_ID	0
Order_Date	0
Customer_ID	0
Product_ID	0
Region	24
Quantity	18
Unit_Price	0
Discount	36
Payment_Mode	18
Rating	48

Task 1: NumPy Introduction & Arrays (10 Marks)

Explanation

Imports NumPy, creates a 12-element array and prints its shape, size and data type. It then builds a one-dimensional and a two-dimensional (3x3) array and compares their dimensions, plus a few other creation methods (zeros, arange, linspace).

Source code (task01_numpy_arrays.py)

```
"""Task 1: NumPy Introduction & Arrays"""
import numpy as np

# Array with 10+ numbers
arr = np.array([12, 45, 7, 23, 56, 89, 34, 21, 67, 90, 15, 42])
print("Array:", arr)
print("Shape:", arr.shape)
print("Size:", arr.size)
print("Data type:", arr.dtype)

# One-dimensional array
arr_1d = np.array([1, 2, 3, 4, 5])
print("\n1-D array:", arr_1d)
print("Dimensions:", arr_1d.ndim, "| Shape:", arr_1d.shape)

# Two-dimensional array
arr_2d = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])
print("\n2-D array:\n", arr_2d)
print("Dimensions:", arr_2d.ndim, "| Shape:", arr_2d.shape,
      "| Size:", arr_2d.size, "| dtype:", arr_2d.dtype)

# Bonus: other ways to create arrays
print("\nZeros:", np.zeros(4))
print("Arange(0,20,4):", np.arange(0, 20, 4))
print("Linspace(0,1,5):", np.linspace(0, 1, 5))
```

Program output

```
Array: [12 45  7 23 56 89 34 21 67 90 15 42]
Shape: (12,)
Size: 12
Data type: int64

1-D array: [1 2 3 4 5]
Dimensions: 1 | Shape: (5,)

2-D array:
 [[1 2 3]
 [4 5 6]
 [7 8 9]]
Dimensions: 2 | Shape: (3, 3) | Size: 9 | dtype: int64

Zeros: [0. 0. 0. 0.]
Arange(0,20,4): [ 0  4  8 12 16]
Linspace(0,1,5): [0.   0.25 0.5  0.75 1.  ]
```

Task 2: NumPy Indexing, Slicing & Reshaping (10 Marks)

Explanation

Accesses elements by positive and negative index, extracts portions with slicing (including step and reverse), selects rows, columns and sub-matrices of a 2-D array, and reshapes a 12-element array into (3,4), (4,3) and (2,2,3), showing original and reshaped arrays.

Source code (task02_indexing_slicing_reshaping.py)

```
"""Task 2: NumPy Indexing, Slicing & Reshaping"""
import numpy as np

arr = np.arange(10, 130, 10)          # 12 elements: 10, 20, ... 120
print("Original 1-D array:", arr)

# Indexing
print("\n--- Indexing ---")
print("First element arr[0]:", arr[0])
print("Fifth element arr[4]:", arr[4])
print("Last element arr[-1]:", arr[-1])

# Slicing
print("\n--- Slicing ---")
print("arr[2:7]  :", arr[2:7])
print("arr[:4]   :", arr[:4])
print("arr[::2]  :", arr[::2])
print("arr[::-1] :", arr[::-1])

# 2-D array
arr_2d = np.array([[11, 12, 13, 14],
                   [21, 22, 23, 24],
                   [31, 32, 33, 34]])
print("\n2-D array:\n", arr_2d)

# Rows and columns
print("\n--- Rows & Columns ---")
print("Row 1 (index 1):", arr_2d[1])
print("Column 2 (index 2):", arr_2d[:, 2])
print("Element [2][3]:", arr_2d[2, 3])
print("Sub-matrix rows 0-1, cols 1-2:\n", arr_2d[0:2, 1:3])

# Reshaping
print("\n--- Reshaping ---")
print("Original shape:", arr.shape)
print("Original array:", arr)
r1 = arr.reshape(3, 4)
print("\nReshaped to (3, 4):\n", r1)
r2 = arr.reshape(4, 3)
print("\nReshaped to (4, 3):\n", r2)
r3 = arr.reshape(2, 2, 3)
print("\nReshaped to (2, 2, 3):\n", r3)
print("\nFlattened back:", r1.flatten())
```

Program output

```
Original 1-D array: [ 10  20  30  40  50  60  70  80  90 100 110 120]

--- Indexing ---
First element arr[0]: 10
Fifth element arr[4]: 50
Last element arr[-1]: 120

--- Slicing ---
arr[2:7]  : [30 40 50 60 70]
arr[:4]   : [10 20 30 40]
arr[::2]  : [ 10  30  50  70  90 110]
arr[::-1] : [120 110 100  90  80  70  60  50  40  30  20  10]

2-D array:
 [[11 12 13 14]
 [21 22 23 24]
 [31 32 33 34]]

--- Rows & Columns ---
Row 1 (index 1): [21 22 23 24]
Column 2 (index 2): [13 23 33]
Element [2][3]: 34
Sub-matrix rows 0-1, cols 1-2:
 [[12 13]
 [22 23]]

--- Reshaping ---
Original shape: (12,)
```

```
Original array: [ 10  20  30  40  50  60  70  80  90 100 110 120]

Reshaped to (3, 4):
 [[ 10  20  30  40]
 [ 50  60  70  80]
 [ 90 100 110 120]]

Reshaped to (4, 3):
 [[ 10  20  30]
 [ 40  50  60]
 [ 70  80  90]
 [100 110 120]]

Reshaped to (2, 2, 3):
 [[[ 10  20  30]
  [ 40  50  60]]

 [[ 70  80  90]
  [100 110 120]]]

Flattened back: [ 10  20  30  40  50  60  70  80  90 100 110 120]
```

Task 3: NumPy Mathematical & Statistical Operations (10 Marks)

Explanation

Uses monthly unit sales of two stores as the numerical dataset. Performs element-wise addition, subtraction, multiplication and division, and computes mean, median, minimum, maximum, standard deviation and sum for each store.

Source code (task03_numpy_math_stats.py)

```
"""Task 3: NumPy Mathematical & Statistical Operations"""
import numpy as np

# Dataset: monthly units sold in two stores (Jan-Dec)
store_a = np.array([120, 135, 150, 160, 145, 170, 180, 175, 190, 200, 210, 230])
store_b = np.array([100, 110, 130, 125, 140, 150, 155, 165, 160, 175, 185, 195])
print("Store A:", store_a)
print("Store B:", store_b)

print("\n=== Mathematical Operations ===")
print("Addition       (A + B):", store_a + store_b)
print("Subtraction    (A - B):", store_a - store_b)
print("Multiplication (A * B):", store_a * store_b)
print("Division       (A / B):", np.round(store_a / store_b, 2))

print("\n=== Statistical Functions (Store A) ===")
print("Mean               :", np.mean(store_a))
print("Median             :", np.median(store_a))
print("Minimum            :", np.min(store_a))
print("Maximum            :", np.max(store_a))
print("Standard Deviation :", round(np.std(store_a), 2))
print("Sum                :", np.sum(store_a))

print("\n=== Statistical Functions (Store B) ===")
print("Mean               :", np.mean(store_b))
print("Median             :", np.median(store_b))
print("Minimum            :", np.min(store_b))
print("Maximum            :", np.max(store_b))
print("Standard Deviation :", round(np.std(store_b), 2))
print("Sum                :", np.sum(store_b))

print("\nBest month for Store A (index):", np.argmax(store_a) + 1)
```

Program output

```
Store A: [120 135 150 160 145 170 180 175 190 200 210 230]
Store B: [100 110 130 125 140 150 155 165 160 175 185 195]

=== Mathematical Operations ===
Addition       (A + B): [220 245 280 285 285 320 335 340 350 375 395 425]
Subtraction    (A - B): [20 25 20 35  5 20 25 10 30 25 25 35]
Multiplication (A * B): [12000 14850 19500 20000 20300 25500 27900 28875 30400 35000 38850 44850]
Division       (A / B): [1.2  1.23 1.15 1.28 1.04 1.13 1.16 1.06 1.19 1.14 1.14 1.18]

=== Statistical Functions (Store A) ===
Mean               : 172.08333333333334
Median             : 172.5
Minimum            : 120
Maximum            : 230
Standard Deviation : 30.85
Sum                : 2065

=== Statistical Functions (Store B) ===
Mean               : 149.16666666666666
Median             : 152.5
Minimum            : 100
Maximum            : 195
Standard Deviation : 28.05
Sum                : 1790

Best month for Store A (index): 12
```

Task 4: Pandas Series & DataFrame (10 Marks)

Explanation

Creates a Series of student marks and a DataFrame of student information. Displays the DataFrame, its column names and index, then adds new columns (Result, Percentage) and shows the updated DataFrame.

Source code (task04_pandas_series_dataframe.py)

```
"""Task 4: Pandas Series & DataFrame"""
import pandas as pd

# Series
marks = pd.Series([85, 92, 78, 66, 90], index=["Amit", "Riya", "Karan", "Neha", "Sara"],
                  name="Maths Marks")
print("Pandas Series:\n", marks)
print("Series mean:", marks.mean())

# DataFrame - student information
data = {
    "Student_ID": [101, 102, 103, 104, 105],
    "Name": ["Amit", "Riya", "Karan", "Neha", "Sara"],
    "Age": [20, 21, 20, 22, 21],
    "Course": ["Economics", "Statistics", "Economics", "Commerce", "Statistics"],
    "Marks": [85, 92, 78, 66, 90],
}
df = pd.DataFrame(data)
print("\nDataFrame:\n", df)
print("\nColumn names:", list(df.columns))
print("Index:", df.index)

# Add new column
df["Result"] = ["Pass" if m >= 70 else "Needs Improvement" for m in df["Marks"]]
df["Percentage"] = df["Marks"] / 100 * 100
print("\nUpdated DataFrame:\n", df)
```

Program output

```
Pandas Series:
 Amit     85
Riya     92
Karan    78
Neha     66
Sara     90
Name: Maths Marks, dtype: int64
Series mean: 82.2

DataFrame:
    Student_ID   Name  Age      Course  Marks
0         101   Amit   20   Economics     85
1         102   Riya   21  Statistics     92
2         103  Karan   20   Economics     78
3         104   Neha   22    Commerce     66
4         105   Sara   21  Statistics     90

Column names: ['Student_ID', 'Name', 'Age', 'Course', 'Marks']
Index: RangeIndex(start=0, stop=5, step=1)

Updated DataFrame:
    Student_ID   Name  Age      Course  Marks             Result  Percentage
0         101   Amit   20   Economics     85               Pass        85.0
1         102   Riya   21  Statistics     92               Pass        92.0
2         103  Karan   20   Economics     78               Pass        78.0
3         104   Neha   22    Commerce     66  Needs Improvement        66.0
4         105   Sara   21  Statistics     90               Pass        90.0
```

Task 5: Reading & Inspecting Data (10 Marks)

Explanation

Reads sales_data.csv (600 orders, 10 columns) and inspects it with head(), tail(), shape, columns, dtypes, info() and describe() to understand structure, types and value ranges.

Source code (task05_reading_inspecting.py)

```
"""Task 5: Reading & Inspecting Data"""
import os
import pandas as pd

pd.set_option("display.width", 110)
pd.set_option("display.max_columns", 20)
BASE = os.path.dirname(os.path.abspath(__file__))

df = pd.read_csv(os.path.join(BASE, "data", "sales_data.csv"))

print("First 5 rows:\n", df.head())
print("\nLast 5 rows:\n", df.tail())
print("\nNumber of rows and columns:", df.shape)
print("Rows:", df.shape[0], "| Columns:", df.shape[1])
print("\nColumn names:", list(df.columns))
print("\nData types:\n", df.dtypes)
print("\ninfo():")
df.info()
print("\ndescribe():\n", df.describe())
```

Program output

```
First 5 rows:
   Order_ID  Order_Date Customer_ID Product_ID   Region  Quantity  Unit_Price  Discount Payment_Mode  Rating
0  ORD0001  2025-01-02        C057        P12     West       2.0       370.0       NaN  Credit Card     3.0
1  ORD0002  2025-01-02        C076        P03     West       6.0      2186.0      0.20          UPI     3.0
2  ORD0003  2025-01-02        C012        P08    South       4.0      4595.0      0.00  Credit Card     NaN
3  ORD0004  2025-01-02        C043        P05  Central       7.0      5159.0      0.15  Credit Card     4.0
4  ORD0005  2025-01-02        C003        P02  Central       1.0     19975.0      0.00  Credit Card     1.0

Last 5 rows:
     Order_ID  Order_Date Customer_ID Product_ID   Region  Quantity  Unit_Price  Discount Payment_Mode  \
595  ORD0596  2025-12-30        C052        P11    North       5.0       918.0      0.00  Net Banking
596  ORD0597  2025-12-31        C014        P09  Central       NaN       223.0      0.15   Debit Card
597  ORD0598  2025-12-31        C010        P05    South       4.0      5159.0      0.10          UPI
598  ORD0599  2025-12-31        C006        P08    North       6.0      4595.0      0.05   Debit Card
599  ORD0600  2025-12-31        C039        P11     East       7.0       918.0      0.00          UPI

     Rating
595     2.0
596     1.0
597     5.0
598     1.0
599     1.0

Number of rows and columns: (600, 10)
Rows: 600 | Columns: 10

Column names: ['Order_ID', 'Order_Date', 'Customer_ID', 'Product_ID', 'Region', 'Quantity', 'Unit_Price', 'Discount',
    'Payment_Mode', 'Rating']

Data types:
 Order_ID            str
Order_Date          str
Customer_ID         str
Product_ID          str
Region              str
Quantity        float64
Unit_Price      float64
Discount        float64
Payment_Mode        str
Rating          float64
dtype: object

info():
<class 'pandas.DataFrame'>
RangeIndex: 600 entries, 0 to 599
Data columns (total 10 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Order_ID      600 non-null    str
 1   Order_Date    600 non-null    str
 2   Customer_ID   600 non-null    str
 3   Product_ID    600 non-null    str
 4   Region        576 non-null    str
 5   Quantity      582 non-null    float64
 6   Unit_Price    600 non-null    float64
```

```
 7   Discount      564 non-null    float64
 8   Payment_Mode  582 non-null    str
 9   Rating        552 non-null    float64
dtypes: float64(4), str(6)
memory usage: 47.0 KB

describe():
          Quantity    Unit_Price    Discount      Rating
count  582.000000    600.000000  564.000000  552.000000
mean     4.036082   8344.793333    0.066223    2.967391
std      1.998382  14448.650295    0.068982    1.391844
min      1.000000    120.000000    0.000000    1.000000
25%      2.000000    370.000000    0.000000    2.000000
50%      4.000000   4595.000000    0.050000    3.000000
75%      6.000000   8210.000000    0.100000    4.000000
max      7.000000  60152.000000    0.200000    5.000000
```

Task 6: Selecting, Filtering & Sorting Data (10 Marks)

Explanation

Selects columns, rows (iloc/loc), filters with a single condition (price) and multiple conditions using & and isin(), and sorts in ascending and descending order, including multi-column sorting.

Source code (task06_select_filter_sort.py)

```
"""Task 6: Selecting, Filtering & Sorting Data"""
import os
import pandas as pd

pd.set_option("display.width", 110)
pd.set_option("display.max_columns", 20)
BASE = os.path.dirname(os.path.abspath(__file__))
df = pd.read_csv(os.path.join(BASE, "data", "sales_data.csv"))

print("--- Select specific columns ---")
print(df[["Order_ID", "Region", "Unit_Price"]].head())

print("\n--- Select specific rows ---")
print("Rows 5 to 9 using iloc:\n", df.iloc[5:10])
print("\nRows using loc (index 0, 2, 4), selected columns:\n",
      df.loc[[0, 2, 4], ["Order_ID", "Product_ID", "Quantity"]])

print("\n--- Filter on a condition (Unit_Price > 20000) ---")
expensive = df[df["Unit_Price"] > 20000]
print(expensive.head())
print("Matching rows:", len(expensive))

print("\n--- Multiple conditions (Region = North AND Quantity >= 5) ---")
multi = df[(df["Region"] == "North") & (df["Quantity"] >= 5)]
print(multi.head())
print("Matching rows:", len(multi))

print("\n--- Multiple conditions with OR (UPI or Cash, Rating >= 4) ---")
either = df[(df["Payment_Mode"].isin(["UPI", "Cash"])) & (df["Rating"] >= 4)]
print(either.head())
print("Matching rows:", len(either))

print("\n--- Sort ascending by Unit_Price ---")
print(df.sort_values("Unit_Price", ascending=True)[
    ["Order_ID", "Product_ID", "Unit_Price"]].head())

print("\n--- Sort descending by Unit_Price, then Quantity ---")
print(df.sort_values(["Unit_Price", "Quantity"], ascending=[False, False])[
    ["Order_ID", "Product_ID", "Unit_Price", "Quantity"]].head())
```

Program output

```
--- Select specific columns ---
  Order_ID   Region  Unit_Price
0  ORD0001     West       370.0
1  ORD0002     West      2186.0
2  ORD0003    South      4595.0
3  ORD0004  Central      5159.0
4  ORD0005  Central     19975.0

--- Select specific rows ---
Rows 5 to 9 using iloc:
   Order_ID  Order_Date Customer_ID Product_ID   Region  Quantity  Unit_Price  Discount Payment_Mode  Rating
5  ORD0006  2025-01-03        C059        P07    North       2.0       643.0      0.00          UPI     3.0
6  ORD0007  2025-01-03        C061        P11  Central       3.0       918.0      0.00  Credit Card     3.0
7  ORD0008  2025-01-04        C062        P01     West       1.0     60152.0      0.15          NaN     5.0
8  ORD0009  2025-01-04        C059        P12  Central       2.0       370.0      0.00          UPI     4.0
9  ORD0010  2025-01-05        C069        P05  Central       7.0      5159.0       NaN          UPI     2.0

Rows using loc (index 0, 2, 4), selected columns:
   Order_ID Product_ID  Quantity
0  ORD0001        P12       2.0
2  ORD0003        P08       4.0
4  ORD0005        P02       1.0

--- Filter on a condition (Unit_Price > 20000) ---
    Order_ID  Order_Date Customer_ID Product_ID Region  Quantity  Unit_Price  Discount Payment_Mode  Rating
7    ORD0008  2025-01-04        C062        P01   West       1.0     60152.0      0.15          NaN     5.0
30   ORD0031  2025-01-16        C051        P01   East       7.0     60152.0      0.00         Cash     4.0
103  ORD0104  2025-03-02        C034        P01  North       7.0     60152.0      0.00  Credit Card     2.0
107  ORD0108  2025-03-06        C032        P01  South       7.0     60152.0      0.00         Cash     1.0
118  ORD0119  2025-03-15        C056        P01   West       6.0     60152.0      0.05   Debit Card     4.0
Matching rows: 36

--- Multiple conditions (Region = North AND Quantity >= 5) ---
   Order_ID  Order_Date Customer_ID Product_ID Region  Quantity  Unit_Price  Discount Payment_Mode  Rating
```

```
47  ORD0048  2025-01-27        C001        P09  North       6.0       223.0      0.05   Debit Card     NaN
54  ORD0055  2025-01-31        C060        P02  North       6.0     19975.0      0.05   Debit Card     4.0
64  ORD0065  2025-02-07        C053        P02  North       5.0     19975.0      0.15  Credit Card     4.0
66  ORD0067  2025-02-08        C075        P03  North       5.0      2186.0      0.00         Cash     1.0
74  ORD0075  2025-02-12        C080        P07  North       5.0       643.0      0.05   Debit Card     3.0
Matching rows: 47

--- Multiple conditions with OR (UPI or Cash, Rating >= 4) ---
   Order_ID  Order_Date Customer_ID Product_ID   Region  Quantity  Unit_Price  Discount Payment_Mode  Rating
8   ORD0009  2025-01-04        C059        P12  Central       2.0       370.0      0.00          UPI     4.0
18  ORD0019  2025-01-10        C055        P02     West       4.0     19975.0      0.10          UPI     4.0
25  ORD0026  2025-01-15        C005        P08     East       7.0      4595.0      0.05          UPI     4.0
30  ORD0031  2025-01-16        C051        P01     East       7.0     60152.0      0.00         Cash     4.0
34  ORD0035  2025-01-18        C062        P02    South       1.0     19975.0      0.00         Cash     5.0
Matching rows: 89

--- Sort ascending by Unit_Price ---
    Order_ID Product_ID  Unit_Price
592  ORD0593        P10       120.0
593  ORD0594        P10       120.0
587  ORD0588        P10       120.0
518  ORD0519        P10       120.0
522  ORD0523        P10       120.0

--- Sort descending by Unit_Price, then Quantity ---
    Order_ID Product_ID  Unit_Price  Quantity
30   ORD0031        P01     60152.0       7.0
103  ORD0104        P01     60152.0       7.0
107  ORD0108        P01     60152.0       7.0
143  ORD0144        P01     60152.0       7.0
187  ORD0188        P01     60152.0       7.0
```

Task 7: Handling Missing Values (10 Marks)

Explanation

Detects missing values with isnull(), counts them per column, removes rows with dropna() and fills values with suitable methods: mode for categorical columns (Region, Payment_Mode), median for Quantity, 0 for Discount and mean for Rating. The dataset is shown before and after, with a short note on why this matters.

Source code (task07_missing_values.py)

```
"""Task 7: Handling Missing Values"""
import os
import pandas as pd

pd.set_option("display.width", 110)
pd.set_option("display.max_columns", 20)
BASE = os.path.dirname(os.path.abspath(__file__))
df = pd.read_csv(os.path.join(BASE, "data", "sales_data.csv"))

print("=== BEFORE handling missing values ===")
print("Shape:", df.shape)
print("\nRows with at least one missing value (first 5):")
print(df[df.isnull().any(axis=1)].head())
print("\nMissing values per column (isnull().sum()):\n", df.isnull().sum())
print("Total missing cells:", df.isna().sum().sum())

# Option 1: drop rows
dropped = df.dropna()
print("\n=== Option 1: dropna() ===")
print("Shape after dropping:", dropped.shape,
      "| rows lost:", df.shape[0] - dropped.shape[0])

# Option 2: fill with suitable values
filled = df.copy()
filled["Region"] = filled["Region"].fillna(filled["Region"].mode()[0])
filled["Payment_Mode"] = filled["Payment_Mode"].fillna(filled["Payment_Mode"].mode()[0])
filled["Quantity"] = filled["Quantity"].fillna(filled["Quantity"].median())
filled["Discount"] = filled["Discount"].fillna(0)
filled["Rating"] = filled["Rating"].fillna(round(filled["Rating"].mean(), 1))

print("\n=== Option 2: fillna() ===")
print("Region/Payment_Mode -> mode | Quantity -> median | Discount -> 0 | Rating -> mean")
print("\nMissing values per column AFTER filling:\n", filled.isnull().sum())
print("Shape after filling:", filled.shape)
print("\nSample of the filled dataset:\n", filled.head())

print("""
Why handling missing data matters in Data Analytics:
 - Missing values bias averages, totals and trends, giving wrong conclusions.
 - Many functions and ML models fail or silently skip rows containing NaN.
 - Dropping too many rows wastes data; filling carelessly adds false information,
   so the method must suit the column (mode for categories, median for skewed numbers).
 - Clean data makes results reliable and reproducible.""")
```

Program output

```
=== BEFORE handling missing values ===
Shape: (600, 10)

Rows with at least one missing value (first 5):
   Order_ID  Order_Date Customer_ID Product_ID   Region  Quantity  Unit_Price  Discount Payment_Mode  Rating
0   ORD0001  2025-01-02        C057        P12     West       2.0       370.0       NaN  Credit Card     3.0
2   ORD0003  2025-01-02        C012        P08    South       4.0      4595.0      0.00  Credit Card     NaN
7   ORD0008  2025-01-04        C062        P01     West       1.0     60152.0      0.15          NaN     5.0
9   ORD0010  2025-01-05        C069        P05  Central       7.0      5159.0       NaN          UPI     2.0
15  ORD0016  2025-01-07        C060        P10     East       3.0       120.0      0.00          UPI     NaN

Missing values per column (isnull().sum()):
 Order_ID         0
Order_Date       0
Customer_ID      0
Product_ID       0
Region          24
Quantity        18
Unit_Price       0
Discount        36
Payment_Mode    18
Rating          48
dtype: int64
Total missing cells: 144

=== Option 1: dropna() ===
Shape after dropping: (472, 10) | rows lost: 128
=== Option 2: fillna() ===
```

```
Region/Payment_Mode -> mode | Quantity -> median | Discount -> 0 | Rating -> mean

Missing values per column AFTER filling:
 Order_ID        0
Order_Date      0
Customer_ID     0
Product_ID      0
Region          0
Quantity        0
Unit_Price      0
Discount        0
Payment_Mode    0
Rating          0
dtype: int64
Shape after filling: (600, 10)

Sample of the filled dataset:
   Order_ID  Order_Date Customer_ID Product_ID   Region  Quantity  Unit_Price  Discount Payment_Mode  Rating
0  ORD0001  2025-01-02        C057        P12     West       2.0       370.0      0.00  Credit Card     3.0
1  ORD0002  2025-01-02        C076        P03     West       6.0      2186.0      0.20          UPI     3.0
2  ORD0003  2025-01-02        C012        P08    South       4.0      4595.0      0.00  Credit Card     3.0
3  ORD0004  2025-01-02        C043        P05  Central       7.0      5159.0      0.15  Credit Card     4.0
4  ORD0005  2025-01-02        C003        P02  Central       1.0     19975.0      0.00  Credit Card     1.0

Why handling missing data matters in Data Analytics:
 - Missing values bias averages, totals and trends, giving wrong conclusions.
 - Many functions and ML models fail or silently skip rows containing NaN.
 - Dropping too many rows wastes data; filling carelessly adds false information,
   so the method must suit the column (mode for categories, median for skewed numbers).
 - Clean data makes results reliable and reproducible.
```

Task 8: Merge, Concatenate, GroupBy & Pivot Table (15 Marks)

Explanation

Merges sales with the products table (Product_ID) and customers table (Customer_ID), concatenates two half-year DataFrames, groups by Category and Region with sum, mean, count, min and max, and builds pivot tables of revenue (Category x Region) and units (Segment x Payment Mode).

Source code (task08_merge_groupby_pivot.py)

```
"""Task 8: Merge, Concatenate, GroupBy & Pivot Table"""
import os
import pandas as pd

pd.set_option("display.width", 110)
pd.set_option("display.max_columns", 20)
BASE = os.path.dirname(os.path.abspath(__file__))
sales = pd.read_csv(os.path.join(BASE, "data", "sales_data.csv"))
products = pd.read_csv(os.path.join(BASE, "data", "products.csv"))
customers = pd.read_csv(os.path.join(BASE, "data", "customers.csv"))

# quick cleaning so aggregates are meaningful
sales["Region"] = sales["Region"].fillna("Unknown")
sales["Quantity"] = sales["Quantity"].fillna(sales["Quantity"].median())
sales["Discount"] = sales["Discount"].fillna(0)

print("=== MERGE ===")
merged = sales.merge(products, on="Product_ID", how="left")
merged = merged.merge(customers, on="Customer_ID", how="left")
merged["Revenue"] = (merged["Quantity"] * merged["Unit_Price"]
                     * (1 - merged["Discount"])).round(2)
print("Merged on Product_ID and Customer_ID. Shape:", merged.shape)
print(merged[["Order_ID", "Product_Name", "Category", "Segment", "Revenue"]].head())

print("\n=== CONCATENATE ===")
h1 = merged[merged["Order_Date"] < "2025-07-01"]
h2 = merged[merged["Order_Date"] >= "2025-07-01"]
print("First half rows:", len(h1), "| Second half rows:", len(h2))
combined = pd.concat([h1, h2], ignore_index=True)
print("After concat rows:", len(combined), "(equals original:", len(combined) == len(merged), ")")

print("\n=== GROUPBY ===")
print("Total revenue by Category:\n", merged.groupby("Category")["Revenue"].sum().round(2))
print("\nMultiple aggregates by Region:")
print(merged.groupby("Region")["Revenue"].agg(["sum", "mean", "count", "min", "max"]).round(2))
print("\nAverage Rating by Category:\n",
      merged.groupby("Category")["Rating"].mean().round(2))

print("\n=== PIVOT TABLE ===")
pivot = pd.pivot_table(merged, values="Revenue", index="Category",
                       columns="Region", aggfunc="sum", margins=True,
                       margins_name="Total", fill_value=0).round(0)
print("Revenue: Category (rows) x Region (columns)\n", pivot)

pivot2 = pd.pivot_table(merged, values="Quantity", index="Segment",
                        columns="Payment_Mode", aggfunc="sum", fill_value=0)
print("\nUnits sold: Segment x Payment_Mode\n", pivot2)
```

Program output

```
=== MERGE ===
Merged on Product_ID and Customer_ID. Shape: (600, 15)
  Order_ID  Product_Name     Category      Segment   Revenue
0  ORD0001  Water Bottle  Accessories     Consumer    740.00
1  ORD0002    Headphones  Electronics     Consumer  10492.80
2  ORD0003     Bookshelf    Furniture  Home Office  18380.00
3  ORD0004  Office Chair    Furniture    Corporate  30696.05
4  ORD0005    Smartphone  Electronics     Consumer  19975.00

=== CONCATENATE ===
First half rows: 285 | Second half rows: 315
After concat rows: 600 (equals original: True )

=== GROUPBY ===
Total revenue by Category:
 Category
Accessories      223316.3
Electronics    15210054.3
Furniture       3538613.0
Stationery        62416.6
Name: Revenue, dtype: float64

Multiple aggregates by Region:
                sum      mean  count    min       max
```

```
Region
Central  2978860.20  26596.97    112  108.0  216547.2
East     4012397.15  36147.72    111  102.0  421064.0
North    3730012.65  33303.68    112  114.0  421064.0
South    4637993.40  37403.17    124  120.0  421064.0
Unknown   368761.50  15365.06     24  200.7  119850.0
West     3306375.30  28259.62    117   96.0  421064.0

Average Rating by Category:
 Category
Accessories    2.72
Electronics    3.11
Furniture      3.08
Stationery     2.72
Name: Rating, dtype: float64

=== PIVOT TABLE ===
Revenue: Category (rows) x Region (columns)
 Region         Central       East      North      South   Unknown       West       Total
Category
Accessories    41264.0    35755.0    54149.0    42009.0   15305.0    34833.0    223316.0
Electronics  2223760.0  3133168.0  3014227.0  3983150.0  256728.0  2599021.0  15210054.0
Furniture     704273.0   827832.0   650364.0   600159.0   94092.0   661893.0   3538613.0
Stationery      9563.0    15641.0    11272.0    12675.0    2637.0    10628.0     62417.0
Total        2978860.0  4012397.0  3730013.0  4637993.0  368762.0  3306375.0  19034400.0

Units sold: Segment x Payment_Mode
 Payment_Mode   Cash  Credit Card  Debit Card  Net Banking    UPI
Segment
Consumer      136.0        300.0       266.0        104.0  412.0
Corporate     125.0        211.0       173.0         88.0  270.0
Home Office    23.0         87.0        52.0         21.0   89.0
```

Task 9: Exporting Data (5 Marks)

Explanation

Cleans the dataset, adds a Revenue column, exports it to output/cleaned_sales_data.csv and verifies the file by re-reading it: file exists, size, shape, columns and zero missing values.

Source code (task09_export.py)

```
"""Task 9: Exporting Data"""
import os
import pandas as pd

pd.set_option("display.width", 110)
pd.set_option("display.max_columns", 20)
BASE = os.path.dirname(os.path.abspath(__file__))
df = pd.read_csv(os.path.join(BASE, "data", "sales_data.csv"))

# Process the dataset
df["Region"] = df["Region"].fillna(df["Region"].mode()[0])
df["Payment_Mode"] = df["Payment_Mode"].fillna(df["Payment_Mode"].mode()[0])
df["Quantity"] = df["Quantity"].fillna(df["Quantity"].median())
df["Discount"] = df["Discount"].fillna(0)
df["Rating"] = df["Rating"].fillna(round(df["Rating"].mean(), 1))
df["Revenue"] = (df["Quantity"] * df["Unit_Price"] * (1 - df["Discount"])).round(2)
print("Processed DataFrame shape:", df.shape)
print("Missing values remaining:", df.isnull().sum().sum())

# Export
out_path = os.path.join(BASE, "output", "cleaned_sales_data.csv")
df.to_csv(out_path, index=False)
print("\nExported to:", os.path.relpath(out_path, BASE))

# Verify
check = pd.read_csv(out_path)
print("\n--- Verification ---")
print("File exists:", os.path.exists(out_path))
print("File size (KB):", round(os.path.getsize(out_path) / 1024, 1))
print("Re-read shape:", check.shape)
print("Same shape as processed data:", check.shape == df.shape)
print("Columns:", list(check.columns))
print("Missing values in exported file:", check.isnull().sum().sum())
print(check.head())
```

Program output

```
Processed DataFrame shape: (600, 11)
Missing values remaining: 0

Exported to: output/cleaned_sales_data.csv

--- Verification ---
File exists: True
File size (KB): 40.4
Re-read shape: (600, 11)
Same shape as processed data: True
Columns: ['Order_ID', 'Order_Date', 'Customer_ID', 'Product_ID', 'Region', 'Quantity', 'Unit_Price', 'Discount',
    'Payment_Mode', 'Rating', 'Revenue']
Missing values in exported file: 0
  Order_ID  Order_Date Customer_ID Product_ID   Region  Quantity  Unit_Price  Discount Payment_Mode  Rating  \
0  ORD0001  2025-01-02        C057        P12     West       2.0       370.0      0.00  Credit Card     3.0
1  ORD0002  2025-01-02        C076        P03     West       6.0      2186.0      0.20          UPI     3.0
2  ORD0003  2025-01-02        C012        P08    South       4.0      4595.0      0.00  Credit Card     3.0
3  ORD0004  2025-01-02        C043        P05  Central       7.0      5159.0      0.15  Credit Card     4.0
4  ORD0005  2025-01-02        C003        P02  Central       1.0     19975.0      0.00  Credit Card     1.0

    Revenue
0    740.00
1  10492.80
2  18380.00
3  30696.05
4  19975.00
```

Task 10: Mini Data Analysis Project: Sales Performance Analysis (10 Marks)

Explanation

End-to-end analysis using NumPy and Pandas: load, inspect, handle missing values, engineer Revenue, Profit and order-value tier (np.where, np.percentile), filter, sort, GroupBy analysis, pivot tables, insights and export of the processed dataset.

Source code (task10_mini_project.py)

```
"""
Task 10: Mini Data Analysis Project - Sales Performance Analysis (2025)
Uses NumPy and Pandas on a sales dataset with product and customer tables.
"""
import os
import numpy as np
import pandas as pd

pd.set_option("display.width", 110)
pd.set_option("display.max_columns", 20)
BASE = os.path.dirname(os.path.abspath(__file__))

# 1. LOAD
print("=" * 70, "\n1. LOADING DATA\n", "=" * 70, sep="")
sales = pd.read_csv(os.path.join(BASE, "data", "sales_data.csv"))
products = pd.read_csv(os.path.join(BASE, "data", "products.csv"))
customers = pd.read_csv(os.path.join(BASE, "data", "customers.csv"))
print("Sales:", sales.shape, "| Products:", products.shape, "| Customers:", customers.shape)

# 2. INSPECT
print("\n" + "=" * 70, "\n2. DATA INSPECTION\n", "=" * 70, sep="")
print(sales.head())
print("\nData types:\n", sales.dtypes)
print("\nSummary statistics:\n", sales.describe().round(2))
print("\nDuplicate rows:", sales.duplicated().sum())

# 3. MISSING VALUES
print("\n" + "=" * 70, "\n3. MISSING VALUES\n", "=" * 70, sep="")
print("Missing before:\n", sales.isnull().sum())
sales["Region"] = sales["Region"].fillna(sales["Region"].mode()[0])
sales["Payment_Mode"] = sales["Payment_Mode"].fillna(sales["Payment_Mode"].mode()[0])
sales["Quantity"] = sales["Quantity"].fillna(sales["Quantity"].median())
sales["Discount"] = sales["Discount"].fillna(0)
sales["Rating"] = sales["Rating"].fillna(round(sales["Rating"].mean(), 1))
print("\nTotal missing after cleaning:", sales.isnull().sum().sum())

# Feature engineering (NumPy + Pandas)
df = sales.merge(products, on="Product_ID", how="left").merge(
    customers, on="Customer_ID", how="left")
df["Order_Date"] = pd.to_datetime(df["Order_Date"])
df["Month"] = df["Order_Date"].dt.month_name()
df["Month_Num"] = df["Order_Date"].dt.month
df["Revenue"] = np.round(df["Quantity"] * df["Unit_Price"] * (1 - df["Discount"]), 2)
df["Profit"] = np.round(df["Revenue"] - df["Quantity"] * df["Cost_Price"], 2)
high_cut = np.percentile(df["Revenue"], 75)
df["Order_Value_Tier"] = np.where(df["Revenue"] >= high_cut, "High", "Regular")
print("\n75th percentile revenue (High-tier cutoff):", round(high_cut, 2))
print(df[["Order_ID", "Product_Name", "Revenue", "Profit", "Order_Value_Tier"]].head())

# 4. SELECT & FILTER
print("\n" + "=" * 70, "\n4. SELECTING & FILTERING\n", "=" * 70, sep="")
high_orders = df[df["Order_Value_Tier"] == "High"]
print("High-value orders:", len(high_orders))
elec_north = df[(df["Category"] == "Electronics") & (df["Region"] == "North")]
print("Electronics orders in North:", len(elec_north))
print(elec_north[["Order_ID", "Product_Name", "Quantity", "Revenue"]].head())

# 5. SORT
print("\n" + "=" * 70, "\n5. SORTING\n", "=" * 70, sep="")
print("Top 5 orders by revenue:")
print(df.sort_values("Revenue", ascending=False)[
    ["Order_ID", "Product_Name", "Region", "Revenue"]].head())

# 6. GROUPBY
print("\n" + "=" * 70, "\n6. GROUPBY ANALYSIS\n", "=" * 70, sep="")
by_cat = df.groupby("Category").agg(Revenue=("Revenue", "sum"), Profit=("Profit", "sum"),
                                    Orders=("Order_ID", "count"),
                                    Avg_Rating=("Rating", "mean")).round(2)
by_cat = by_cat.sort_values("Revenue", ascending=False)
print(by_cat)
```

```
by_region = df.groupby("Region")["Revenue"].sum().sort_values(ascending=False).round(2)
print("\nRevenue by Region:\n", by_region)
by_prod = df.groupby("Product_Name")["Revenue"].sum().sort_values(ascending=False).round(2)
print("\nTop 5 products by revenue:\n", by_prod.head())
by_pay = df["Payment_Mode"].value_counts(normalize=True).mul(100).round(1)
print("\nPayment mode share (%):\n", by_pay)
by_seg = df.groupby("Segment")["Revenue"].sum().sort_values(ascending=False).round(2)
print("\nRevenue by Segment:\n", by_seg)
by_month = df.groupby(["Month_Num", "Month"])["Revenue"].sum().round(2)
print("\nMonthly revenue:\n", by_month)

# 7. PIVOT TABLE
print("\n" + "=" * 70, "\n7. PIVOT TABLES\n", "=" * 70, sep="")
pv = pd.pivot_table(df, values="Revenue", index="Category", columns="Region",
                    aggfunc="sum", margins=True, margins_name="Total").round(0)
print("Revenue by Category x Region:\n", pv)
pv2 = pd.pivot_table(df, values="Profit", index="Segment", columns="Category",
                     aggfunc="mean").round(0)
print("\nAverage profit per order, Segment x Category:\n", pv2)

# 8. INSIGHTS
print("\n" + "=" * 70, "\n8. KEY INSIGHTS\n", "=" * 70, sep="")
total_rev, total_profit = df["Revenue"].sum(), df["Profit"].sum()
best_month = by_month.idxmax()[1]
worst_month = by_month.idxmin()[1]
disc_qty = df.groupby(df["Discount"] > 0)["Quantity"].mean().round(2)
margin = (df.groupby("Category")["Profit"].sum() / df.groupby("Category")["Revenue"].sum()
          * 100).round(1).sort_values(ascending=False)
print(f"1. Total revenue is Rs {total_rev:,.0f} with total profit Rs {total_profit:,.0f} "
      f"(overall margin {total_profit / total_rev * 100:.1f}%).")
print(f"2. {by_cat.index[0]} is the top category, contributing "
      f"{by_cat['Revenue'].iloc[0] / total_rev * 100:.1f}% of revenue.")
print(f"3. {by_region.index[0]} is the best region (Rs {by_region.iloc[0]:,.0f}); "
      f"{by_region.index[-1]} is the weakest (Rs {by_region.iloc[-1]:,.0f}).")
print(f"4. {by_prod.index[0]} is the best-selling product by revenue; "
      f"top 3 products give {by_prod.head(3).sum() / total_rev * 100:.1f}% of revenue.")
print(f"5. {best_month} has the highest monthly revenue and {worst_month} the lowest.")
print(f"6. {by_pay.index[0]} is the most used payment mode ({by_pay.iloc[0]}% of orders).")
print(f"7. Category with best profit margin: {margin.index[0]} ({margin.iloc[0]}%); "
      f"lowest: {margin.index[-1]} ({margin.iloc[-1]}%).")
print(f"8. Average units per order: discounted {disc_qty[True]} vs "
      f"non-discounted {disc_qty[False]}.")
print(f"9. {by_seg.index[0]} customers generate the most revenue "
      f"({by_seg.iloc[0] / total_rev * 100:.1f}%).")

# 9. EXPORT
print("\n" + "=" * 70, "\n9. EXPORT\n", "=" * 70, sep="")
out = os.path.join(BASE, "output", "project_cleaned_sales_analysis.csv")
df.drop(columns=["Month_Num"]).to_csv(out, index=False)
print("Saved:", os.path.relpath(out, BASE), "| shape:", pd.read_csv(out).shape)
```

Program output

```
======================================================================
1. LOADING DATA
======================================================================
Sales: (600, 10) | Products: (12, 4) | Customers: (80, 2)

======================================================================
2. DATA INSPECTION
======================================================================
  Order_ID  Order_Date Customer_ID Product_ID   Region  Quantity  Unit_Price  Discount Payment_Mode  Rating
0  ORD0001  2025-01-02        C057        P12     West       2.0       370.0       NaN  Credit Card     3.0
1  ORD0002  2025-01-02        C076        P03     West       6.0      2186.0      0.20          UPI     3.0
2  ORD0003  2025-01-02        C012        P08    South       4.0      4595.0      0.00  Credit Card     NaN
3  ORD0004  2025-01-02        C043        P05  Central       7.0      5159.0      0.15  Credit Card     4.0
4  ORD0005  2025-01-02        C003        P02  Central       1.0     19975.0      0.00  Credit Card     1.0

Data types:
 Order_ID            str
Order_Date          str
Customer_ID         str
Product_ID          str
Region              str
Quantity        float64
Unit_Price      float64
Discount        float64
Payment_Mode        str
Rating          float64
dtype: object

Summary statistics:
        Quantity  Unit_Price  Discount  Rating
count    582.00      600.00    564.00  552.00
mean       4.04     8344.79      0.07    2.97
std        2.00    14448.65      0.07    1.39
min        1.00      120.00      0.00    1.00
25%        2.00      370.00      0.00    2.00
50%        4.00     4595.00      0.05    3.00
```

```
75%        6.00     8210.00      0.10    4.00
max        7.00    60152.00      0.20    5.00

Duplicate rows: 0

======================================================================
3. MISSING VALUES
======================================================================
Missing before:
 Order_ID         0
Order_Date        0
Customer_ID       0
Product_ID        0
Region           24
Quantity         18
Unit_Price        0
Discount         36
Payment_Mode     18
Rating           48
dtype: int64

Total missing after cleaning: 0

75th percentile revenue (High-tier cutoff): 31960.0
  Order_ID  Product_Name   Revenue   Profit Order_Value_Tier
0  ORD0001  Water Bottle    740.00   240.00          Regular
1  ORD0002    Headphones  10492.80  1492.80          Regular
2  ORD0003     Bookshelf  18380.00  5580.00          Regular
3  ORD0004  Office Chair  30696.05  1296.05          Regular
4  ORD0005    Smartphone  19975.00  4975.00          Regular

======================================================================
4. SELECTING & FILTERING
======================================================================
High-value orders: 151
Electronics orders in North: 40
   Order_ID Product_Name  Quantity    Revenue
54  ORD0055   Smartphone       6.0  113857.50
64  ORD0065   Smartphone       5.0   84893.75
66  ORD0067   Headphones       5.0   10930.00
79  ORD0080   Smartwatch       2.0   12682.00
94  ORD0095   Smartwatch       2.0   11413.80

======================================================================
5. SORTING
======================================================================
Top 5 orders by revenue:
    Order_ID Product_Name Region   Revenue
143  ORD0144       Laptop   East  421064.0
346  ORD0347       Laptop   West  421064.0
30   ORD0031       Laptop   East  421064.0
345  ORD0346       Laptop   East  421064.0
103  ORD0104       Laptop  North  421064.0

======================================================================
6. GROUPBY ANALYSIS
======================================================================
                Revenue     Profit  Orders  Avg_Rating
Category
Electronics  15210054.3  3490554.3     206        3.10
Furniture     3538613.0   892863.0     189        3.07
Accessories    223316.3    51116.3     103        2.74
Stationery      62416.6    10306.6     102        2.75

Revenue by Region:
 Region
South      5006754.90
East       4012397.15
North      3730012.65
West       3306375.30
Central    2978860.20
Name: Revenue, dtype: float64

Top 5 products by revenue:
 Product_Name
Laptop        7678402.80
Smartphone    5939566.25
Study Desk    1857512.50
Smartwatch    1254566.85
Bookshelf      854899.75
Name: Revenue, dtype: float64

Payment mode share (%):
 Payment_Mode
UPI            35.0
Credit Card    26.3
Debit Card     18.8
Cash           11.0
Net Banking     8.8
Name: proportion, dtype: float64

Revenue by Segment:
```

```
 Segment
Consumer       9198139.2
Corporate      7191049.5
Home Office    2645211.5
Name: Revenue, dtype: float64

Monthly revenue:
 Month_Num  Month
1          January      1624219.50
2          February      783763.20
3          March        1858701.80
4          April        1392942.25
5          May          1246515.50
6          June         2713713.95
7          July         2478967.15
8          August       2257562.20
9          September    1211132.30
10         October      1326480.80
11         November     1003710.90
12         December     1136690.65
Name: Revenue, dtype: float64

======================================================================
7. PIVOT TABLES
======================================================================
Revenue by Category x Region:
 Region         Central       East      North      South       West       Total
Category
Accessories    41264.0    35755.0    54149.0    57314.0    34833.0    223316.0
Electronics  2223760.0  3133168.0  3014227.0  4239878.0  2599021.0  15210054.0
Furniture     704273.0   827832.0   650364.0   694251.0   661893.0   3538613.0
Stationery      9563.0    15641.0    11272.0    15312.0    10628.0     62417.0
Total        2978860.0  4012397.0  3730013.0  5006755.0  3306375.0  19034400.0

Average profit per order, Segment x Category:
 Category     Accessories  Electronics  Furniture  Stationery
Segment
Consumer           499.0      16659.0     4696.0       111.0
Corporate          452.0      17645.0     4356.0        77.0
Home Office        617.0      16136.0     6095.0       107.0

======================================================================
8. KEY INSIGHTS
======================================================================
1. Total revenue is Rs 19,034,400 with total profit Rs 4,444,840 (overall margin 23.4%).
2. Electronics is the top category, contributing 79.9% of revenue.
3. South is the best region (Rs 5,006,755); Central is the weakest (Rs 2,978,860).
4. Laptop is the best-selling product by revenue; top 3 products give 81.3% of revenue.
5. June has the highest monthly revenue and February the lowest.
6. UPI is the most used payment mode (35.0% of orders).
7. Category with best profit margin: Furniture (25.2%); lowest: Stationery (16.5%).
8. Average units per order: discounted 3.86 vs non-discounted 4.26.
9. Consumer customers generate the most revenue (48.3%).

======================================================================
9. EXPORT
======================================================================
Saved: output/project_cleaned_sales_analysis.csv | shape: (600, 18)
```

Key Findings / Insights

1. Total revenue is Rs 19,034,400 with total profit Rs 4,444,840 (overall margin 23.4%).

2. Electronics is the top category, contributing 79.9% of revenue.

3. South is the best region (Rs 5,006,755); Central is the weakest (Rs 2,978,860).

4. Laptop is the best-selling product by revenue; top 3 products give 81.3% of revenue.

5. June has the highest monthly revenue and February the lowest.

6. UPI is the most used payment mode (35.0% of orders).

7. Category with best profit margin: Furniture (25.2%); lowest: Stationery (16.5%).

8. Average units per order: discounted 3.86 vs non-discounted 4.26.

9. Consumer customers generate the most revenue (48.3%).

Recommendations: protect Electronics stock and pricing, grow the weaker regions, plan inventory around the peak month and review discounting. *The dataset is synthetic, so findings demonstrate the method, not real business results.*

Submission Contents

Google Drive link: [paste your shared link here - set access to "Anyone with the link: Viewer"]

File / folder	Purpose
generate_dataset.py	Dataset generator
task01_numpy_arrays.py	Source code for the task
task02_indexing_slicing_reshaping.py	Source code for the task
task03_numpy_math_stats.py	Source code for the task
task04_pandas_series_dataframe.py	Source code for the task
task05_reading_inspecting.py	Source code for the task
task06_select_filter_sort.py	Source code for the task
task07_missing_values.py	Source code for the task
task08_merge_groupby_pivot.py	Source code for the task
task09_export.py	Source code for the task
task10_mini_project.py	Source code for the task
data/	sales_data.csv, products.csv, customers.csv
output/	cleaned_sales_data.csv (Task 9), project_cleaned_sales_analysis.csv (Task 10)